SMART-X PROJECT COMPLETION REPORT AND TECHNOLOGY RECOMMENDATIONS 1
1. Project Completion Report 1
1.1 Project Overview and Completion 1
1.2 Architectural Challenges and Solutions 1
Integrating the ASP.NET Core backend with the WPF frontend 1
Implementing historical telemetry storage using an AVL tree 1
Processing telemetry using FIFO queues 1
Representing the mesh network using a weighted graph 1
Visualising the network topology 1
Implementing command history and undo functionality 1
1.3 Debugging and Problem-Solving Approaches 1
1.4 Key Learnings and Skills Acquired 1
Data structures and algorithm selection 1
Understanding algorithmic complexity 1
Developing layered .NET applications 1
Security and data protection 1
Testing and development workflow 1
1.5 Additional Implementation Features 1
1.5.1 Generics and Operator Overloading 1
1.5.2 Arrays and Recursive Deployment Validation 1
1.5.3 Dashboard Monitoring and Engagement 1
2. Technology Recommendations and Future Scaling 1
2.1 MQTT and an IoT Message Broker 1
2.2 Google Cloud Platform for Distributed Hosting 1
2.3 Docker, Kubernetes and GitHub Actions 1
2.4 OpenTelemetry and Centralised Monitoring 1
2.5 LLM Integration for Telemetry Analysis 1

2.6 Home Assistant Integration....1
2.7 Time-Series Database Technology....1
3. Recommended Future Development Roadmap....1
4. Conclusion....1
References....1

SMART-X PROJECT COMPLETION REPORT AND TECHNOLOGY RECOMMENDATIONS

Prepared by: Tameez Lambat

Project: Smart-X IoT Ecosystem

Technology Stack: C#, .NET 10, ASP.NET Core Web API, WPF, XAML, Supabase Auth, xUnit and GitHub Actions

1. Project Completion Report

1.1 Project Overview and Completion

The Smart-X project involved developing an IoT ecosystem capable of managing simulated connected devices, processing telemetry, examining historical sensor readings and representing device connectivity through a mesh network.

The solution was developed using a layered architecture consisting of three primary components:

- **SmartX.Shared:** Contains reusable models, enumerations, generic collections, telemetry data structures and mesh-network graph logic.
- **SmartXAPI:** Provides backend functionality through ASP.NET Core Web API endpoints and services for telemetry, device management, commands, network topology and routing.
- **SmartXWPF:** Provides the Windows desktop interface through which users interact with the system, monitor sensors, examine historical telemetry and visualise network connectivity.

The project also incorporates automated testing through xUnit, authentication integration through Supabase Auth, AES-GCM encryption for attachment data and GitHub Actions for automated test execution. These technologies provide supporting functionality for application development, authentication, data protection and software

verification (Microsoft, n.d.-a; Microsoft, n.d.-b; Microsoft, n.d.-c; Supabase, n.d.; Dworkin, 2007; xUnit.net, n.d.; GitHub, n.d.).

A significant achievement was bringing these separate components together into a functioning application rather than implementing each requirement independently. The shared library provides reusable functionality, the API exposes that functionality to external clients, and the WPF application presents it through an accessible interface.

1.2 Architectural Challenges and Solutions

Integrating the ASP.NET Core backend with the WPF frontend

One of the main architectural challenges was connecting the C# backend to the WPF desktop application while maintaining separation of responsibilities.

The frontend needed to retrieve sensor information, request historical telemetry, display network topology and execute operations such as BFS, DFS and shortest-path calculations. Placing these operations directly inside the WPF application would have created unnecessary coupling between the interface and the underlying business logic.

This was addressed by separating the application into the shared library, API and desktop frontend. The WPF application communicates with the ASP.NET Core API through HTTP requests and JSON responses. The API handles the service operations, while the frontend focuses on displaying results and responding to user interaction. This approach aligns with the separation of application responsibilities supported by the .NET and ASP.NET Core development frameworks (Microsoft, n.d.-a; Microsoft, n.d.-b).

Shared models and enumerations help maintain consistency between the projects.

This approach demonstrated the importance of API contracts and separation of concerns. The frontend does not need to know how a graph algorithm is implemented; it only needs to request the relevant operation and interpret the returned result.

Implementing historical telemetry storage using an AVL tree

Another challenge involved storing and retrieving historical telemetry efficiently.

A standard list would require examining individual entries when searching for readings across a timestamp range. Smart-X therefore uses a generic AVL tree, `TelemetryAvlTree<T>`, to maintain timestamp-ordered telemetry data.

Implementing the AVL tree required handling node heights, balance factors and the four rotation cases:

- Left-left rotation.
- Right-right rotation.
- Left-right rotation.
- Right-left rotation.

These rotations ensure that the tree remains balanced as new telemetry arrives. The implementation also accommodates multiple telemetry packets sharing the same timestamp and supports date-range queries with optional device MAC-address filtering.

The main difficulty was maintaining the balancing rules while ensuring that range queries returned the correct readings. This was addressed through separate insertion, balancing and query operations, supported by automated tests.

The `QueryRange` operation takes advantage of the tree's timestamp ordering to avoid examining branches that cannot contain matching readings. A conventional unsorted list would generally need to examine every entry, resulting in O(n) search time. The balanced AVL structure allows relevant sections to be located more efficiently, with typical range-query complexity of O(log n + k), where k represents the number of matching results, subject to the cost of processing duplicate timestamps and applying filters. These performance considerations reflect the broader importance of selecting suitable data structures and analysing their algorithmic complexity (Cormen et al., 2022).

The topology interface also allows users to select a sensor and specify a start and end date to retrieve historical readings for the chosen period.

This demonstrated why balanced search trees are valuable when an application requires ordered data and efficient historical retrieval.

Processing telemetry using FIFO queues

Telemetry ingestion introduced a different requirement: incoming readings needed to be processed in their arrival order.

A FIFO queue was selected because it ensures that the first reading added is the first reading removed for processing. Queue-based processing is a standard data-structure approach for managing items in arrival order (Cormen et al., 2022).

This separates the arrival of telemetry from its processing. It also provides a foundation for handling batches of readings without requiring the ingestion operation and processing operation to happen simultaneously.

The challenge was understanding that the queue and AVL tree serve different purposes. The queue supports ordered processing, while the AVL tree supports historical organisation and querying.

Representing the mesh network using a weighted graph

The network topology required representing devices and their connections in a way that supported routing and connectivity analysis.

Smart-X uses a weighted graph in `MeshNetWorkGraph`, with devices represented as vertices and connections represented as weighted edges. Weighted graphs are useful for modelling networks in which connections have associated costs or distances (Cormen et al., 2022).

The graph uses nested dictionaries to store adjacency information and a `HashSet` to track offline devices.

Several algorithms were implemented to solve different network problems:

- **Breadth-First Search (BFS):** Determines which devices are reachable from a starting device and explores the network level by level (GeeksforGeeks, 2026).
- **Depth-First Search (DFS):** Explores connected devices by following paths as deeply as possible before backtracking (GeeksforGeeks, 2026).
- **Dijkstra's algorithm:** Calculates a minimum-weight route between a selected device and the gateway (GeeksforGeeks, 2026).
- **Prim's algorithm:** Produces a minimum spanning tree for the connected component reachable from the selected starting device (GeeksforGeeks, 2026).

These algorithms demonstrate different ways of analysing graph connectivity, traversal and weighted routes (Cormen et al., 2022).

A key challenge was ensuring that these algorithms behaved correctly when devices were offline or disconnected.

The system distinguishes between devices that are offline and devices that are online but unreachable from the gateway. This is important because an online device can still be isolated from the rest of the network.

The implementation also required understanding that a minimum spanning tree and a shortest path are not interchangeable.

Dijkstra's algorithm determines the lowest-cost route between two specific devices. It considers the weights of individual connections and uses a priority queue to process the most promising routes. This is useful when determining the most efficient communication route between a sensor and the gateway (GeeksforGeeks, 2026a; Cormen et al., 2022).

Prim's algorithm solves a different problem. Instead of finding a route between two devices, it constructs a minimum spanning tree that connects the reachable network component while minimising the total weight of the selected connections (GeeksforGeeks, 2026b; Cormen et al., 2022).

This distinction is important because the shortest route between two devices does not necessarily produce the minimum total cost for connecting an entire network.

Visualizing the network topology

Calculating network connectivity was only part of the task. The results also needed to be presented in a way that users could understand.

The WPF topology renderer uses BFS-based levels to position reachable devices according to their hop distance from the gateway.

The renderer also displays connection weights and differentiates between gateways, relays, sensors, offline devices and unreachable devices.

One important consideration was ensuring that the visual representation accurately reflected the underlying graph. The BFS layout represents hop-count levels, not weighted shortest-path distances. Keeping these concepts separate prevents users from interpreting visual placement as a measure of connection cost (GeeksforGeeks, 2026c; Cormen et al., 2022).

The application also compares the cost of the configured connections against the cost of the minimum spanning tree, demonstrating how graph algorithms can assist with network planning.

Implementing command history and undo functionality

Smart-X also includes simulated device commands through `CommandService`.

Commands are validated according to the device category, and the service maintains device simulation state and a history of executed commands.

A `Stack<UndoEntry>` was used to support undo functionality. Before a reversible command changes a device's state, a copy of the previous state is pushed onto the stack. When an undo is requested, the most recent entry is popped and the previous state is restored.

The READ command is deliberately excluded from the undo stack because it records an event rather than changing device state.

This feature demonstrated the difference between maintaining a command history and maintaining the information needed to reverse operations. The history uses a list, while the undo mechanism uses a stack because undo operations follow last-in, first-out behaviour (Cormen et al., 2022).

1.3 Debugging and Problem-Solving Approaches

Throughout development, a recurring challenge was determining which application layer was responsible when something did not behave as expected.

Instead of treating every issue as a frontend or backend problem, the development process involved examining the flow of data through the application.

For example, when a topology result was incorrect, the investigation could be broken down into:

1. Confirming that the expected devices and connections were present.
2. Checking the online/offline state of the devices.
3. Verifying the graph algorithm's result.
4. Confirming that the API returned the expected information.
5. Checking that the WPF application interpreted and displayed the response correctly.

A similar approach was used for telemetry and historical queries. The stored data, timestamp ordering, query boundaries and returned results needed to be checked independently.

Automated testing was particularly important for algorithmic functionality. The graph tests include independent verification of minimum spanning tree and shortest-path results rather than relying solely on the production implementation to validate itself. Unit

testing provides a repeatable way to verify expected behaviour as an application changes (xUnit.net, n.d.).

This approach helped develop a more systematic debugging process: isolate the problem, verify assumptions, reproduce the issue and test the correction.

1.4 Key Learnings and Skills Acquired

Data structures and algorithm selection

One of the most important lessons was learning to select data structures according to the problem being solved.

Smart-X provided practical examples of several structures:

- `SmartCollection<T>` demonstrated generic programming and encapsulation through a custom collection built around `List<T>`.
- FIFO queues supported ordered telemetry processing.
- AVL trees supported timestamp-based historical data retrieval.
- Dictionaries supported efficient device and connection lookups.
- HashSets supported membership checks and offline-device tracking.
- Priority queues supported weighted graph algorithms.
- Stacks supported undo functionality.

The project reinforced that data structures should not be selected simply because they are available in a programming language. Their suitability depends on the operations, access patterns and performance requirements of the application (Cormen et al., 2022).

Understanding algorithmic complexity

The assignment also developed a stronger understanding of Big-O notation and how algorithmic choices influence performance.

For example, BFS and DFS are commonly associated with O(V + E) traversal complexity when implemented with suitable adjacency-list structures. AVL tree operations maintain logarithmic height through balancing, while Dijkstra and Prim use priority queues to efficiently process weighted connections (Cormen et al., 2022; GeeksforGeeks, 2026a; GeeksforGeeks, 2026b; GeeksforGeeks, 2026c).

An additional learning was that actual implementation details matter. Extra searches, filtering and repeated lookups can increase the cost beyond the theoretical complexity of the underlying algorithm.

This encouraged a more critical approach to analysing code rather than simply assigning textbook complexity values.

Developing layered .NET applications

Working with separate shared, API and WPF projects improved my understanding of software architecture in the .NET ecosystem.

I gained practical experience with:

- Creating reusable classes and shared models.
- Separating business logic from presentation logic.
- Exposing functionality through ASP.NET Core API endpoints.
- Communicating between applications using HTTP and JSON.
- Managing application state through services.
- Using asynchronous operations when communicating with the backend.

The project demonstrated how a well-separated architecture makes applications easier to maintain, test and extend. These concepts are supported by the wider .NET and ASP.NET Core development frameworks (Microsoft, n.d.-a; Microsoft, n.d.-b).

Security and data protection

The authentication and encryption requirements introduced important security considerations.

Supabase Auth provided an authentication integration, while AES-GCM encryption was implemented for attachment data (Supabase, n.d.; Dworkin, 2007).

This developed my understanding of the difference between authentication and data encryption. Authentication determines whether a user is permitted to access the system, while encryption protects data from being read without the appropriate key.

AES-GCM is an authenticated encryption mode that provides confidentiality and authentication of encrypted data (Dworkin, 2007).

The implementation also highlighted the importance of configuration management and protecting sensitive values. Encryption keys and credentials must not be exposed in source control, and encrypted data depends on retaining the correct key material.

Testing and development workflow

The xUnit test project and GitHub Actions workflow introduced a more structured approach to software verification.

Instead of relying exclusively on manually running the application, important algorithmic functionality can be tested repeatedly. xUnit provides the testing framework used to organise and execute the automated tests, while GitHub Actions supports automated workflows for building and testing software (xUnit.net, n.d.; GitHub, n.d.).

The project demonstrated how automated workflows can help identify problems early, verify changes and reduce the likelihood of introducing regressions.

Overall, the most valuable learning was developing the ability to break complex engineering problems into smaller, testable components. This is particularly important in an application such as Smart-X, where a single feature may involve a data structure, service, API endpoint and frontend component.

1.5 Additional Implementation Features

1.5.1 Generics and Operator Overloading

The Smart-X system uses generic programming to support different types of sensor readings through a reusable packet structure. This allows the same packet implementation to handle decimal values, whole numbers and Boolean on/off readings without requiring separate packet classes for each data type.

Generics preserve the original data type, reducing the need for unnecessary conversions to strings or general-purpose objects. This improves type safety and makes the implementation easier to maintain (Microsoft, n.d.-a).

Operator overloading is also implemented to allow the + operator to combine readings according to the packet's defined addition behaviour. This makes operations on sensor readings more intuitive while keeping the logic within the relevant type.

1.5.2 Arrays and Recursive Deployment Validation

The system uses jagged arrays to organise sensor readings into groups that can contain different numbers of elements. Unlike a rectangular two-dimensional array, a jagged array allows each inner array to have its own length.

This is useful when different sensors or groups produce different quantities of readings.

Recursive deployment validation is implemented in `DeploymentValidationService`. The `ValidateDeploymentRecursively()` method checks each deployment node and then calls itself for every child node. This allows the entire deployment hierarchy to be validated using the same method.

The recursion includes several stopping conditions:

- Null nodes are rejected.
- Nodes with empty or whitespace-only names are rejected.
- The maximum permitted depth is 10.
- A root node without children is rejected.
- If any child fails validation, the parent hierarchy is also considered invalid.

These stopping conditions prevent invalid structures from being accepted and limit excessively deep recursive calls.

This approach demonstrates how recursion can simplify the traversal and validation of hierarchical structures, while a depth limit helps protect against excessively deep or malformed deployment paths.

1.5.3 Dashboard Monitoring and Engagement

The Smart-X dashboard combines monitoring information to help users identify and respond to potential sensor and network issues. Alerts are ranked according to severity, active errors are tracked, recommendations are presented, and the live graph provides a visual representation of incoming sensor readings.

These features help users identify abnormal behaviour without having to inspect every sensor individually. For example, if a sensor stops sending readings, the absence of recent telemetry can help indicate a possible communication or device failure. Alerts and recommendations provide additional context for investigating the issue.

The combination of live visualisation, alert prioritisation and error tracking makes the dashboard useful for ongoing system monitoring rather than simply displaying raw sensor data.

2. Technology Recommendations and Future Scaling

The current Smart-X implementation provides a foundation for further development. However, a production-scale IoT ecosystem would benefit from additional technologies that improve telemetry ingestion, scalability, monitoring, interoperability and automated deployment.

The following recommendations are proposed future enhancements and are not presented as existing project functionality.

2.1 MQTT and an IoT Message Broker

Recommendation: Integrate an MQTT broker such as Eclipse Mosquitto or EMQX.

MQTT is a lightweight publish/subscribe messaging protocol designed for IoT and machine-to-machine communication. Devices could publish readings to topics organised by device identifier, location or measurement category (Banks et al., 2019).

Smart-X could subscribe to these topics through a .NET background service and feed incoming messages into its existing telemetry processing pipeline.

Benefits:

- Reduces the need for devices to communicate directly with individual API endpoints.
- Supports large numbers of connected devices.
- Separates telemetry producers from consumers.
- Provides a lightweight communication mechanism for constrained devices.
- Allows multiple services to consume the same telemetry stream.

Compatibility with .NET:

The ASP.NET Core ecosystem can integrate with MQTT through compatible .NET client libraries. A background worker could subscribe to the broker, validate messages and pass them to the existing telemetry services.

This would allow Smart-X to retain its current domain logic while introducing a more realistic device communication architecture.

2.2 Google Cloud Platform for Distributed Hosting

Recommendation: Evaluate Google Cloud services such as Cloud SQL, Pub/Sub and Google Kubernetes Engine.

As the number of connected devices increases, storing and processing all telemetry through a single application instance may become a limitation.

Google Cloud Pub/Sub could provide a managed messaging layer for incoming telemetry, while Cloud SQL could provide managed relational database hosting. For larger analytical workloads, BigQuery could be considered (Google Cloud, n.d.).

Benefits:

- Managed infrastructure reduces database administration.
- Pub/Sub can absorb bursts of incoming telemetry.
- Cloud SQL provides managed backups and database availability features.
- BigQuery can support large-scale historical analysis.
- Cloud infrastructure provides a path towards scaling beyond a single server.

Compatibility with .NET:

Google provides client libraries and APIs that can be consumed by C# applications. Smart-X could introduce cloud services through dedicated integration classes or background workers without making the shared domain library dependent on a specific cloud provider.

This would preserve the existing layered architecture while allowing the backend to scale independently.

2.3 Docker, Kubernetes and GitHub Actions

Recommendation: Expand containerisation and CI/CD using Docker, Kubernetes where justified, and GitHub Actions.

Docker can package the Smart-X API and supporting background services into consistent deployment units (Docker, n.d.).

Docker Compose would be suitable for local development, allowing the API, database and MQTT broker to run together.

For a larger deployment, Kubernetes or Google Kubernetes Engine could manage multiple application instances (Kubernetes, n.d.).

GitHub Actions should continue to automate builds and tests, with additional steps for dependency scanning, container image creation and deployment (GitHub, n.d.).

Benefits:

- Consistent environments across development and production.
- Automated testing before deployment.
- Easier deployment and rollback.
- Horizontal scaling of API and worker services.
- Improved release reliability.

Compatibility with .NET:

.NET applications are well suited to containerisation. ASP.NET Core applications can run inside Docker containers, and GitHub Actions can execute .NET restore, build and test commands (Microsoft, n.d.-b; Docker, n.d.; GitHub, n.d.).

Kubernetes should be introduced only when the operational requirements justify its additional complexity. For the current project, Docker and GitHub Actions provide a more practical starting point.

Current Project Scope:

Docker containerisation was considered as part of the project's deployment strategy but was not implemented in the current submission. This was a deliberate scope-management decision, as the mandatory project requirements already involved a substantial implementation workload. With the submission deadline approaching, priority was given to completing, integrating and testing the required application functionality rather than introducing additional optional deployment infrastructure. Docker and container orchestration therefore remain recommendations for improvement.

2.4 OpenTelemetry and Centralized Monitoring

Recommendation: Integrate OpenTelemetry with a monitoring platform such as Grafana and Prometheus.

An IoT application requires more than knowing whether the API is running. Operators need visibility into how telemetry moves through the system and where failures or delays occur.

OpenTelemetry could be used to collect:

- API request latency.
- Telemetry ingestion rates.
- Queue processing delays.
- Failed telemetry operations.
- Authentication failures.
- Offline-device counts.
- Database query performance.

Benefits:

- Faster identification of performance bottlenecks.
- Improved visibility across API and background services.
- Easier diagnosis of failures involving multiple components.
- Historical monitoring of system performance.
- More informed capacity planning.

Compatibility with .NET:

OpenTelemetry provides .NET instrumentation and integration options for ASP.NET Core and background services. This makes it a natural addition to the existing Smart-X architecture (OpenTelemetry, n.d.-a).

OpenTelemetry metrics can also be exported for use with Prometheus and Grafana, providing a foundation for visualising application performance and operational measurements (OpenTelemetry, n.d.-b).

Distributed tracing would be particularly useful if MQTT and cloud messaging were introduced, as it would help trace a reading from its original ingestion through to storage and API retrieval.

2.5 LLM Integration for Telemetry Analysis

Recommendation: Introduce an optional Large Language Model API, such as Azure OpenAI, to summarize daily telemetry alerts and operational events.

As the number of connected devices grows, operators may receive large volumes of alerts and sensor readings. Reviewing every event individually can become inefficient.

An LLM-powered service could analyze pre-aggregated telemetry and alert information and produce summaries describing:

- Repeated device failures.
- Unusual sensor readings.
- Frequent offline events.
- Potential patterns across multiple devices.
- Daily or weekly operational summaries.

For example, Smart-X could generate a report indicating that several devices in a particular zone experienced repeated connectivity interruptions during a specific period.

Benefits:

- Reduces the time required to review large alert volumes.
- Makes technical information easier to understand.
- Helps operators identify recurring patterns.
- Provides a more accessible reporting interface.

Compatibility with .NET:

The ASP.NET Core backend could communicate with an LLM API through HTTPS or a supported .NET SDK. Azure OpenAI provides a potential managed service for introducing this type of functionality (Microsoft, n.d.-d).

However, the model should not be treated as the source of truth. Summaries must be based on validated telemetry, clearly distinguish observed facts from generated interpretations and retain references to the underlying alerts.

Sensitive data should be minimised before being sent to external services. Any consequential device action should require explicit validation and human approval.

2.6 Home Assistant Integration

Recommendation: Explore integration with Home Assistant.

Home Assistant is an open-source home automation platform that supports a wide range of devices and integrations (Home Assistant, n.d.).

Smart-X could integrate with Home Assistant through MQTT, its REST API or supported event interfaces.

This would allow selected Smart-X device states and sensor readings to be shared with an established automation ecosystem.

Benefits:

- Greater interoperability with existing IoT devices.
- Access to established automation functionality.
- Additional ways to visualize and interact with sensor data.
- Reduced need to implement every possible automation integration independently.

Compatibility with .NET:

The Smart-X API could communicate with Home Assistant through HTTP-based interfaces, while MQTT could provide a shared messaging mechanism (Home Assistant, n.d.; Banks et al., 2019).

The integration should use secured credentials and clearly defined permissions to prevent unauthorized device control.

2.7 Time-Series Database Technology

Recommendation: Evaluate TimescaleDB for production telemetry storage if the volume of historical readings increases significantly.

Although the AVL tree is useful for demonstrating balanced-tree operations and supporting timestamp-based queries within the application, a production deployment may eventually need persistent storage capable of handling millions of telemetry readings.

A time-series database provides functionality designed specifically for timestamped measurements. TimescaleDB is built on PostgreSQL and provides time-series capabilities for storing and querying this type of data (Timescale, n.d.).

Benefits:

- Efficient historical time-range queries.
- Aggregation of readings over hourly, daily and monthly periods.
- Data retention and archival policies.
- Improved handling of large telemetry datasets.
- Better support for analytical dashboards.

Compatibility with .NET:

TimescaleDB is built on PostgreSQL and can be accessed through established .NET database providers (Timescale, n.d.).

The existing AVL tree could remain part of the application for in-memory processing and algorithmic functionality, while the database handles long-term persistence and large-scale historical queries.

3. Recommended Future Development Roadmap

The technologies should be introduced progressively rather than all at once.

Priority	Recommendation	Main objective
1	OpenTelemetry and centralised monitoring	Establish visibility into performance and failures.
2	MQTT broker and .NET background worker	Introduce scalable device-to-platform communication.
3	Docker and expanded GitHub Actions workflows	Improve deployment consistency and automation.
4	Cloud messaging and managed database hosting	Support increased telemetry volume and availability.

5	Time-series database	Improve long-term telemetry storage and analysis.
6	LLM-based alert summaries	Improve operator efficiency and reporting.

This order prioritises reliability and observability before introducing more complex infrastructure. Monitoring should be established first so that future scaling decisions can be based on actual measurements rather than assumptions.

4. Conclusion

The Smart-X project provided practical experience in developing a multi-layered IoT application using the C# and .NET ecosystem.

The most significant engineering challenge was integrating data structures, algorithms, backend services and a desktop frontend into a coherent system. The use of a shared library, ASP.NET Core Web API and WPF application helped separate responsibilities and provided a maintainable foundation.

Implementing the AVL tree, FIFO telemetry queue and weighted graph demonstrated how theoretical data structures and algorithms can solve practical application problems (Cormen et al., 2022). The additional implementation of command history, undo functionality, authentication, encryption and automated testing expanded the project beyond basic algorithm development (Dworkin, 2007; Supabase, n.d.; xUnit.net, n.d.; GitHub, n.d.).

The project also improved my understanding of software architecture, API integration, algorithmic complexity, debugging, security and automated testing. The implementation of generics, operator overloading, jagged arrays and recursive validation further demonstrated how programming concepts can be applied to practical IoT requirements.

Most importantly, it reinforced the importance of understanding how individual components interact within a complete application.

Looking forward, MQTT, managed cloud infrastructure, OpenTelemetry, containerisation, time-series storage and optional LLM-based analysis provide realistic opportunities to evolve Smart-X into a more scalable and production-oriented IoT

ecosystem (Banks et al., 2019; Google Cloud, n.d.; OpenTelemetry, n.d.-a; Docker, n.d.; Timescale, n.d.; Microsoft, n.d.-d).

These recommendations build on the architecture already established rather than requiring the application to be redesigned from the beginning. They provide a practical path towards supporting larger device networks, more reliable telemetry processing, improved monitoring and richer operator functionality.

References

Banks, A., Briggs, E., Borgendale, K. and Gupta, R. (2019) *MQTT Version 5.0*. OASIS Standard. Available at: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html (Accessed: 11 November 2026).

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge, MA: MIT Press.

Docker (n.d.) *Docker documentation*. Available at: https://docs.docker.com/ (Accessed: 11 November 2026).

Dworkin, M. (2007) *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-38D (Accessed: 1 November 2026).

GeeksforGeeks (2026a) *Dijkstra's algorithm*. 25 October 2026. Available at: https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/ (Accessed: 1 November 2026).

GeeksforGeeks (2026b) *Prim's Algorithm for Minimum Spanning Tree (MST)*. 2 November 2026. Available at: https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/ (Accessed: 3 November 2026).

GeeksforGeeks (2026c) *Difference between BFS and DFS*. 1 November 2026. Available at: https://www.geeksforgeeks.org/dsa/difference-between-bfs-and-dfs/ (Accessed: 3 November 2026).

GitHub (n.d.) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 11 November 2026).

Google Cloud (n.d.) *Pub/Sub documentation*. Available at: https://docs.cloud.google.com/pubsub/docs (Accessed: 11 November 2026).

Home Assistant (n.d.) *Home Assistant developer documentation*. Available at: https://developers.home-assistant.io/ (Accessed: 11 November 2026).

Kubernetes (n.d.) *Kubernetes documentation*. Available at: https://kubernetes.io/docs/ (Accessed: 11 November 2026).

Microsoft (n.d.-a) *.NET documentation*. Available at: https://learn.microsoft.com/en-us/dotnet/ (Accessed: 11 November 2026).

Microsoft (n.d.-b) *ASP.NET Core documentation*. Available at: https://learn.microsoft.com/en-us/aspnet/core/ (Accessed: 11 November 2026).

Microsoft (n.d.-c) *Windows Presentation Foundation (WPF) documentation*. Available at: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/ (Accessed: 11 November 2026).

Microsoft (n.d.-d) *Azure OpenAI documentation*. Available at: https://learn.microsoft.com/en-us/azure/ai-services/openai/ (Accessed: 11 November 2026).

OpenTelemetry (n.d.-a) *OpenTelemetry .NET documentation*. Available at: https://opentelemetry.io/docs/languages/dotnet/ (Accessed: 11 November 2026).

OpenTelemetry (n.d.-b) *Export to Prometheus and Grafana*. Available at: https://opentelemetry.io/docs/languages/dotnet/metrics/getting-started-prometheus-grafana/ (Accessed: 11 November 2026).

Supabase (n.d.) *Supabase Auth documentation*. Available at: https://supabase.com/docs/guides/auth (Accessed: 11 November 2026).

Timescale (n.d.) *Timescale documentation*. Available at: https://docs.timescale.com/ (Accessed: 11 November 2026).

xUnit.net (n.d.) *xUnit.net documentation*. Available at: https://xunit.net/ (Accessed: 11 November 2026).

SMART-X PROJECT COMPLETION REPORT AND TECHNOLOGY RECOMMENDATIONS 1

1. Project Completion Report 1

1.1 Project Overview and Completion 1
1.2 Architectural Challenges and Solutions 1
Integrating the ASP.NET Core backend with the WPF frontend 1
Implementing historical telemetry storage using an AVL tree 1
Processing telemetry using FIFO queues 1
Representing the mesh network using a weighted graph 1
Visualising the network topology 1
Implementing command history and undo functionality 1
1.3 Debugging and Problem-Solving Approaches 1
1.4 Key Learnings and Skills Acquired 1
Data structures and algorithm selection 1
Understanding algorithmic complexity 1
Developing layered .NET applications 1
Security and data protection 1
Testing and development workflow 1
1.5 Additional Implementation Features 1
1.5.1 Generics and Operator Overloading 1
1.5.2 Arrays and Recursive Deployment Validation 1
1.5.3 Dashboard Monitoring and Engagement 1
2. Technology Recommendations and Future Scaling 1
2.1 MQTT and an IoT Message Broker 1
2.2 Google Cloud Platform for Distributed Hosting 1
2.3 Docker, Kubernetes and GitHub Actions 1
2.4 OpenTelemetry and Centralised Monitoring 1
2.5 LLM Integration for Telemetry Analysis 1
2.6 Home Assistant Integration 1
2.7 Time-Series Database Technology 1
3. Recommended Future Development Roadmap 1
4. Conclusion 1
References 1